FIAP / ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Fase 6 - Navegando pelo mundo DevOps

# ESG Compliance API

## Documentação técnica

Governança e compliance ambiental
Integração contínua, containers e entrega automatizada

---


INTEGRANTES

Paulo Ricardo
RM564600

Pedro Joaquim Garceis Paes
RM565674


São Paulo | 13 de outubro de 2026

Java 21 · Spring Boot · Oracle Free · Docker · GitHub Actions

01 / VISÃO GERAL

# Propósito e escopo

O problema de negócio e a organização da entrega técnica.

A ESG Compliance API centraliza o controle de licenças ambientais, auditorias internas e alertas de conformidade. O projeto apoia a governança ao reunir informações operacionais que podem ser consultadas por diferentes perfis de acesso, com regras de validação e persistência em banco relacional.

Nesta fase, o foco é tornar o ciclo de entrega reproduzível: compilar e testar a aplicação, gerar uma imagem Docker, validar a integração com Oracle e organizar a implantação em staging e produção por meio do GitHub Actions.

| Módulo | Responsabilidade |
|---|---|
| Licenças ambientais | Criar, listar, consultar, atualizar e excluir licenças; garantir código único e validade posterior à emissão. |
| Auditorias internas | Registrar auditorias associadas a uma licença e validar a data de agendamento. |
| Alertas de conformidade | Registrar alertas, consultar pendências e resolver alertas de forma controlada. |
| Dashboard | Consolidar licenças, vencimentos próximos, alertas abertos e auditorias agendadas. |

## Mapa do documento

| Seção | Conteúdo | Página |
|---|---|---|
| Arquitetura | Camadas, tecnologias e persistência | 3 |
| Pipeline CI/CD | Gatilhos, cinco etapas e artefatos | 4 |
| Containerização e orquestração | Dockerfile, Compose e ambientes | 5-6 |
| Qualidade e evidências | Testes, HTTP, Swagger e GitHub | 7-12 |
| Operação e fechamento | Desafios, reprodução e checklist | 13-15 |

**Resultado comprovado**
Execução #2 do GitHub Actions concluída com as cinco etapas aprovadas: 21 testes sem falhas, integração com Oracle, imagem no GHCR e deploys em staging e produção. A API local também passou nas dez verificações HTTP registradas.

02 / ARQUITETURA

# Da requisição ao banco

Separação de responsabilidades e componentes efetivamente presentes no código.

Os controllers expõem os recursos REST; os services concentram regras de negócio; os repositories usam Spring Data JPA para persistência. DTOs delimitam os dados de entrada e saída. O tratamento global de exceções padroniza as respostas de erro.

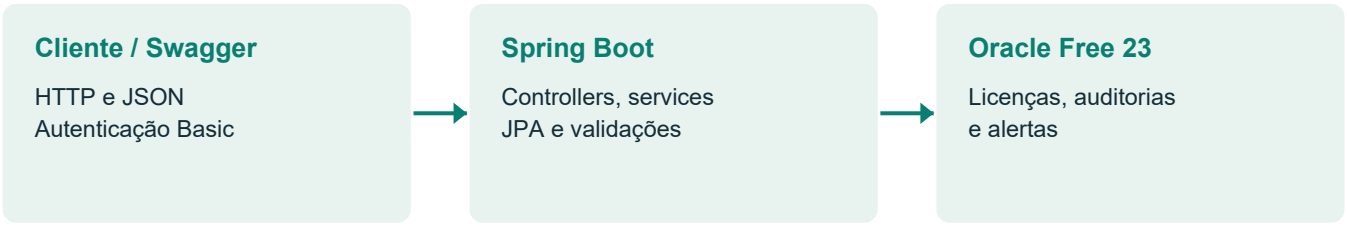



Figura 1. Fluxo lógico da solução. A aplicação e o Oracle se comunicam pela rede esg-net no Docker Compose.

## Tecnologias e versões declaradas

| Componente | Uso no projeto |
|---|---|
| Java 21 + Maven | Compilação e empacotamento. O container de build usa Maven 3.9.9 com Eclipse Temurin 21. |
| Spring Boot 3.3.5 | Web, Data JPA, Validation, Security e Actuator, conforme pom.xml. |
| Oracle Database Free 23 | Imagem gvenzl/oracle-free:23-slim-faststart; driver ojdbc11 23.4.0.24.05. |
| Flyway | V1 cria tabelas, chaves e índices; V2 insere os dados demonstrativos. |
| Springdoc OpenAPI 2.6.0 | Documentação interativa e especificação dos endpoints. |
| JUnit 5 / Mockito / MockMvc | Testes de regras de negócio, respostas HTTP e permissões. |

## Modelo de dados

A tabela **environmental_license** é a entidade central. **internal_audit** e **compliance_alert** possuem chaves estrangeiras para uma licença. O código da licença tem restrição de unicidade. O Hibernate usa **ddl-auto: validate**, enquanto as mudanças de estrutura são versionadas pelo Flyway.

Fonte: pom.xml; src/main/java/br/com/projetof/esg/; src/main/resources/db/migration/.

03 / PIPELINE CI/CD

# Cinco etapas de entrega

Implementação em .github/workflows/ci-cd.yml e deploy.yml.

O workflow principal é disparado por push ou pull request para **main** e também permite execução manual. Pull requests executam build/testes e integração. A publicação da imagem e os deploys dependem de um evento diferente de pull request na branch main.

01 **Build e testes**
mvn -B clean verify; resumo Surefire, relatórios XML e artefato JAR.

02 **Integração**
Docker Compose inicia API + Oracle; smoke test verifica a stack real.

03 **Publicar imagem**
Build Docker e envio ao GHCR com tags sha-<commit> e latest.

04 **Deploy staging**
Workflow reutilizável inicia o ambiente e executa o smoke test.

05 **Deploy produção**
Depende do sucesso de staging e recebe a mesma referência de imagem.

## Artefatos produzidos

**relatorios-testes** preserva os relatórios Surefire; **esg-compliance-api-jar** preserva o pacote Java. No modo runner, **evidencias-staging** e **evidencias-production** guardam o status dos containers, health, info e logs da aplicação por 30 dias.

> **Execução e aprovação**
> As cinco etapas foram concluídas na execução #2, commit 35720db. Produção avançou automaticamente após staging: não houve aprovação manual nesta execução. Required reviewers é uma opção de proteção do GitHub Environment, dependente da configuração e do plano.

A tag baseada no commit permite rastrear a versão. Como tags de registry podem ser sobrescritas, fixar a imagem por digest é uma melhoria futura para garantir identidade imutável.

04 / CONTAINERIZAÇÃO

# Uma imagem para executar

Build separado do runtime, usuário dedicado e verificação de saúde.

O Dockerfile usa dois estágios. O primeiro contém Maven e JDK para compilar o código. O segundo contém JRE Alpine e o JAR necessário à execução. Os testes são executados antes no job de build; o empacotamento dentro da imagem usa -DskipTests.

```
FROM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q -B dependency:go-offline
COPY src ./src
RUN mvn -q -B -DskipTests clean package

FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY --from=build /app/target/esg-compliance-api-1.0.0.jar app.jar
RUN chown app:app app.jar
USER app
```

Trecho do Dockerfile; o arquivo completo inclui labels, APP_VERSION, JAVA_OPTS, EXPOSE, HEALTHCHECK e ENTRYPOINT.

| Decisão | Benefício e comportamento |
|---|---|
| Build em dois estágios | Ferramentas de compilação permanecem fora da imagem final. |
| Cache de dependências | Copiar pom.xml antes do código permite reaproveitar a camada de dependências. |
| Usuário app | O processo Java é executado sem usuário root. |
| Healthcheck | Consulta /actuator/health a cada 30 s, com timeout de 5 s, start-period de 90 s e 5 tentativas. |
| Configuração externa | Variáveis de ambiente definem conexão, perfil, versão, senhas e opções de memória da JVM. |

O .dockerignore exclui target, .git, .env, docs e scripts do contexto de build. Não foi medida a redução de tamanho da imagem nesta validação.

05 / ORQUESTRAÇÃO

# Ambientes com Docker Compose

Arquivo base compartilhado, override local e configuração por ambiente.

O arquivo base define os serviços **app** e **oracle-db**, a rede bridge **esg-net** e o volume **oracle-data**. O depends_on com service_healthy aguarda a disponibilidade do Oracle antes de iniciar a API. Hikari e Flyway também têm configuração para tolerar a inicialização do banco.

```
services:
  app:
    depends_on:
      oracle-db:
        condition: service_healthy
    networks: [esg-net]
  oracle-db:
    volumes:
      - oracle-data:/opt/oracle/oradata
    healthcheck:
      test: ["CMD", "healthcheck.sh"]
```

| Aspecto | Local | Staging / produção |
|---|---|---|
| Configuração | Base + override automático | Base com -f explícito no runner |
| Imagem da API | Build na máquina | Referência publicada no GHCR |
| Banco no host | Porta 1521 pelo override | Sem publicação da porta do banco |
| Porta da API | 8080 | 8081 / 8080, configuráveis |
| Credenciais | Valores locais de demonstração | Secrets ou valores aleatórios no runner |
| Duração | Mantido até ser encerrado | Runner: limitada à execução do job |

> **Modo escolhido para esta entrega**
> O modo runner dispensa uma VM para a demonstração do fluxo. Staging e production são ambientes temporários: não oferecem endereço público persistente nem banco preservado entre jobs. O projeto também contém uma opção de deploy via SSH, que não foi utilizada nesta validação.

A condição service_healthy controla a prontidão na inicialização; ela não implementa recuperação automática da aplicação nem rollback. Referência: documentação Docker, Control startup and shutdown order in Compose [R1].

06 / QUALIDADE

# Testes e controle de acesso

O que é coberto pela suíte e o que foi efetivamente verificado.

O projeto contém **21 métodos de teste** distribuídos em cinco classes. Os testes de services usam Mockito; os de controller usam MockMvc com Spring Security e services simulados. A integração com o Oracle é verificada separadamente pelo smoke test da stack.

| Classe | Testes | Cenários principais |
|---|---|---|
| LicenseServiceTest | 6 | Criação válida, datas inválidas, código duplicado, conflito no update, ausência e filtro. |
| AlertServiceTest | 4 | Criação, resolução, alerta já resolvido e recurso ausente. |
| AuditServiceTest | 2 | Agendamento futuro e rejeição de data passada. |
| DashboardServiceTest | 1 | Agregação dos indicadores retornados pelos repositories. |
| LicenseControllerSecurityTest | 8 | 401, senha incorreta, leitura do analista, 403 na escrita, 201, 400, 404 e dashboard. |

**Execução Maven comprovada**
GitHub Actions #2, commit 35720db: 21 testes, 0 falhas, 0 erros e 0 ignorados. Os cinco XML originais do artefato relatorios-testes acompanham a entrega. O log registra BUILD SUCCESS.

## Perfis de acesso implementados

| Perfil | Permissões |
|---|---|
| ADMIN | Leitura geral e operações de licenças, auditorias e alertas. |
| AUDITOR | Leitura geral; criação de auditorias e alertas; resolução de alertas. |
| ANALYST | Consultas e dashboard; não pode criar licenças. |

As senhas são verificadas com BCrypt e a autenticação é HTTP Basic. Os usuários são mantidos em memória. Para exposição pública, seriam necessários HTTPS e uma gestão de identidades adequada. O health e a documentação são públicos por configuração.

07 / EVIDÊNCIAS

# Validação atual da API

Coleta local em 2026-10-13 18:31:02 (UTC-03). Base: http://localhost:8080.

Foram executadas dez verificações HTTP na instância local já em funcionamento. Todas retornaram o status esperado. As respostas brutas e o registro de coleta acompanham o projeto em docs/evidencias/local/.

| Recurso | Identidade | HTTP | Resultado |
|---|---|---|---|
| /actuator/health | sem credenciais | 200 | OK |
| /actuator/info | sem credenciais | 200 | OK |
| /api/licenses | sem credenciais | 401 | OK |
| /api/licenses | senha incorreta | 401 | OK |
| /api/dashboard/summary | analyst | 200 | OK |
| /api/licenses | admin | 200 | OK |
| /api/audits | auditor | 200 | OK |
| /api/alerts/open | analyst | 200 | OK |
| /v3/api-docs | sem credenciais | 200 | OK |
| /swagger-ui.html | sem credenciais | 200 | OK |

```
GET /actuator/health
{"status":"UP","groups":["liveness","readiness"]}

GET /actuator/info
{"app":{"name":"esg-compliance-api",
        "environment":"local","version":"local"}}
```

No acesso ao Swagger, o cliente seguiu o redirecionamento até a página final, que retornou HTTP 200. Sem credenciais ou com senha incorreta, /api/licenses retornou HTTP 401.

> **Escopo desta verificação**
> Estes resultados comprovam a resposta da API local e os controles exercitados. A versão informada é local, sem identificador de commit; a coleta não comprova execução no GitHub, publicação de imagem ou testes de escrita com banco real.

07 / EVIDÊNCIAS

# Documentação e dados

Captura real da aplicação, acompanhada de uma resposta autenticada.

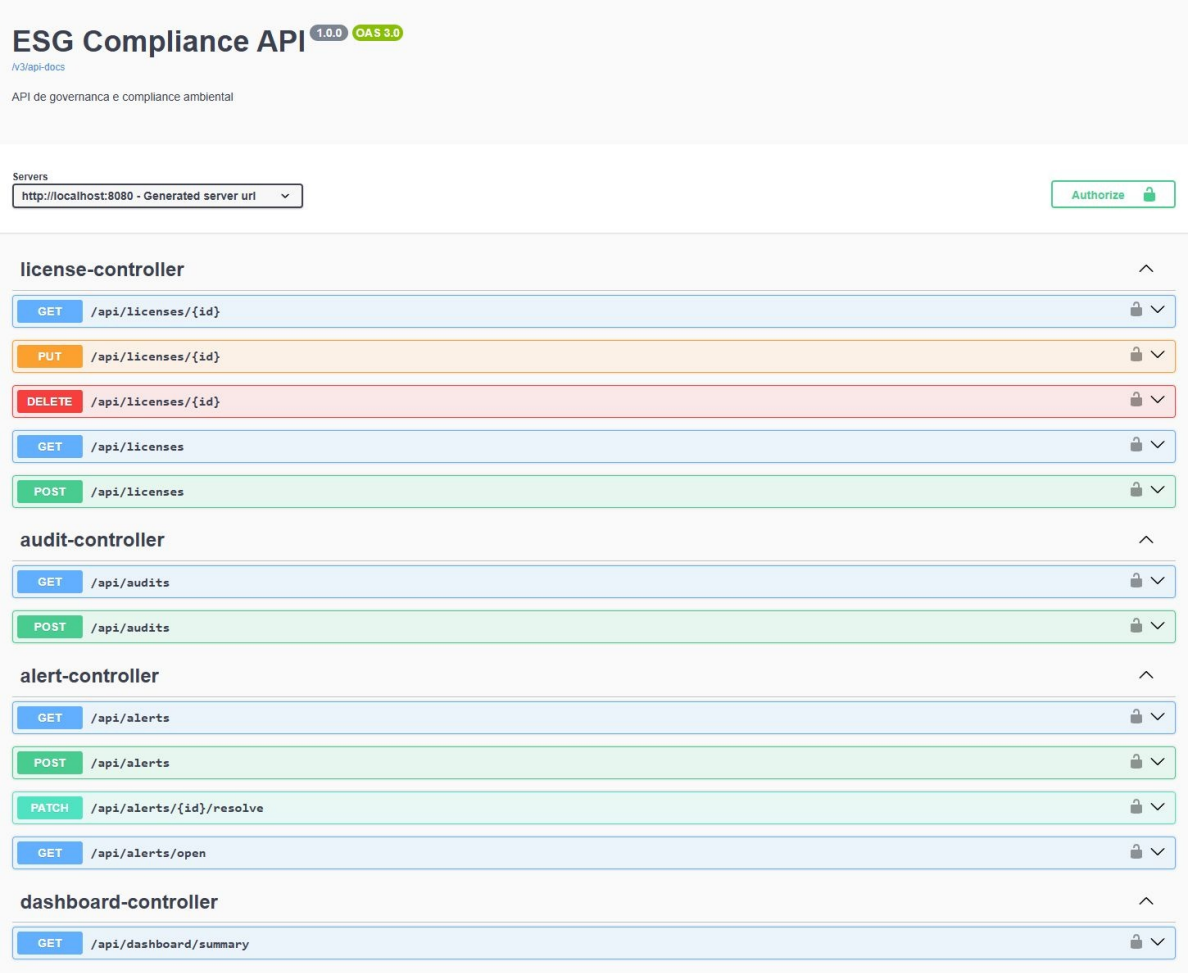



Figura 2. Swagger da ESG Compliance API: licenças, auditorias, alertas e dashboard. Recorte da captura de 13/10/2026; endereço local /swagger-ui/index.html.

## Dashboard autenticado - perfil ANALYST

| Indicador | Valor observado |
|---|---|
| Licenças totais / ativas | 1 / 1 |
| Vencimentos nos próximos 30 dias | 0 |
| Alertas abertos / auditorias agendadas | 1 / 1 |

A consulta com ADMIN retornou a licença LIC-2026-001, da empresa demonstrativa EcoLog Brasil. Os dados são provenientes da carga inicial, não de uma operação empresarial real.

07 / EVIDÊNCIAS

# Pipeline executado no GitHub

Execução #2 de 13/10/2026; repositório privado PauloOliver12/Projeto-ESG-DevOps.

**Abrir execução completa no GitHub Actions** · Commit **35720db097f427cdaa9678af685a9d498d8d70a9**. Os registros desta seção pertencem à mesma execução.

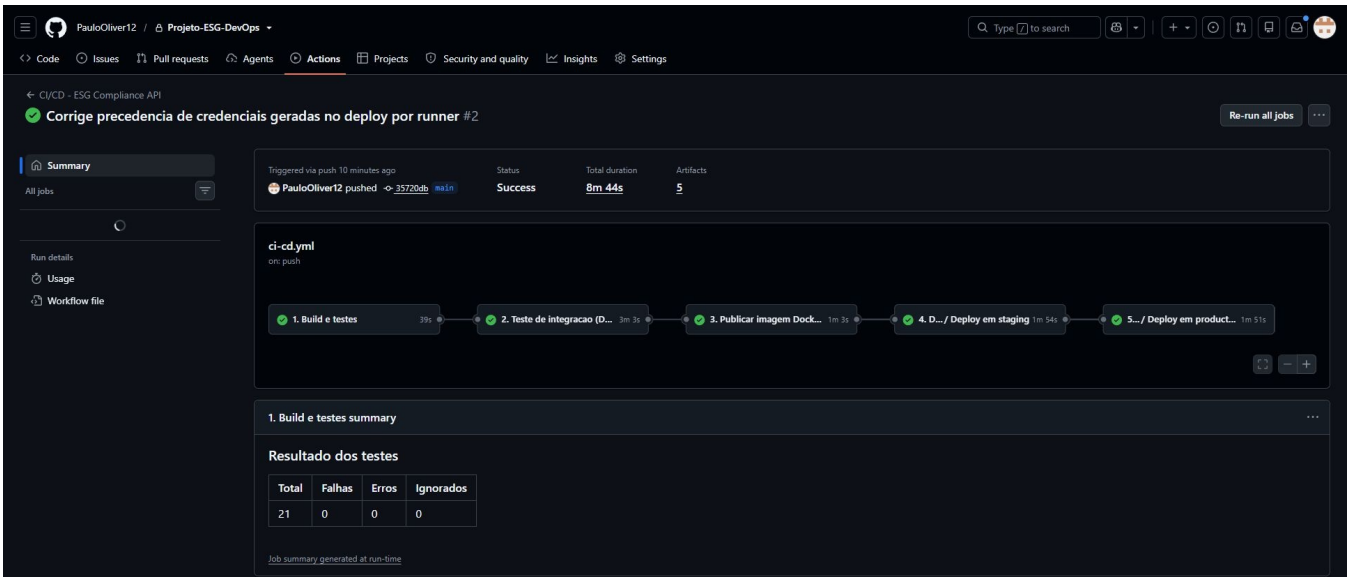



Figura 3. Captura real da execução concluída: cinco etapas aprovadas e resumo dos testes. A imagem original está em docs/prints/02-pipeline-github.jpg.

| Etapa | Resultado | Comprovação |
|---|---|---|
| Build e testes | Sucesso | 21 testes; 0 falhas; 0 erros; 0 ignorados |
| Integração / Oracle | Sucesso | Health UP, 401, Swagger e dashboard autenticado |
| Imagem no GHCR | Sucesso | esg-compliance-api:sha-35720db |
| Deploy staging | Sucesso | Smoke test e evidências do ambiente staging |
| Deploy produção | Sucesso | Smoke test e evidências do ambiente production |

O smoke test de integração verificou a stack real, separadamente dos testes com mocks. Seu log registra ambos os containers como healthy e as duas migrações do Flyway aplicadas. As linhas relevantes foram preservadas em docs/evidencias/github/integration.log.

07 / EVIDÊNCIAS

# Staging e produção comprovados

Mesma referência de imagem; configurações e bancos isolados em runners temporários.

Os dois jobs fizeram pull da imagem publicada, iniciaram API + Oracle e passaram no smoke test: health UP, identificação correta do ambiente, 401 sem login, Swagger disponível e dashboard autenticado. Os arquivos abaixo foram baixados dos artefatos oficiais.

```
STAGING  /actuator/info
{"app":{"name":"esg-compliance-api","environment":"staging","version":"sha-35720db"}}

PRODUCTION  /actuator/info
{"app":{"name":"esg-compliance-api","environment":"production","version":"sha-35720db"}}
```




Figura 4. Recorte dos resumos reais no GitHub: modo runner, imagem publicada e ambiente retornado pela API. A captura original acompanha a entrega.

Os artefatos evidencias-staging e evidencias-production contêm health, info, status dos containers e logs da API. Os arquivos foram preservados em docs/evidencias/github/.

> **O que o deploy demonstra**
> A implantação e as verificações ocorreram em ambos os ambientes durante os jobs. O runner é descartável: esta entrega não mantém uma aplicação pública permanente nem uma VM Azure.

07 / EVIDÊNCIAS

# Imagem publicada e rastreável

Registro privado no GitHub Container Registry e preservação dos resultados.

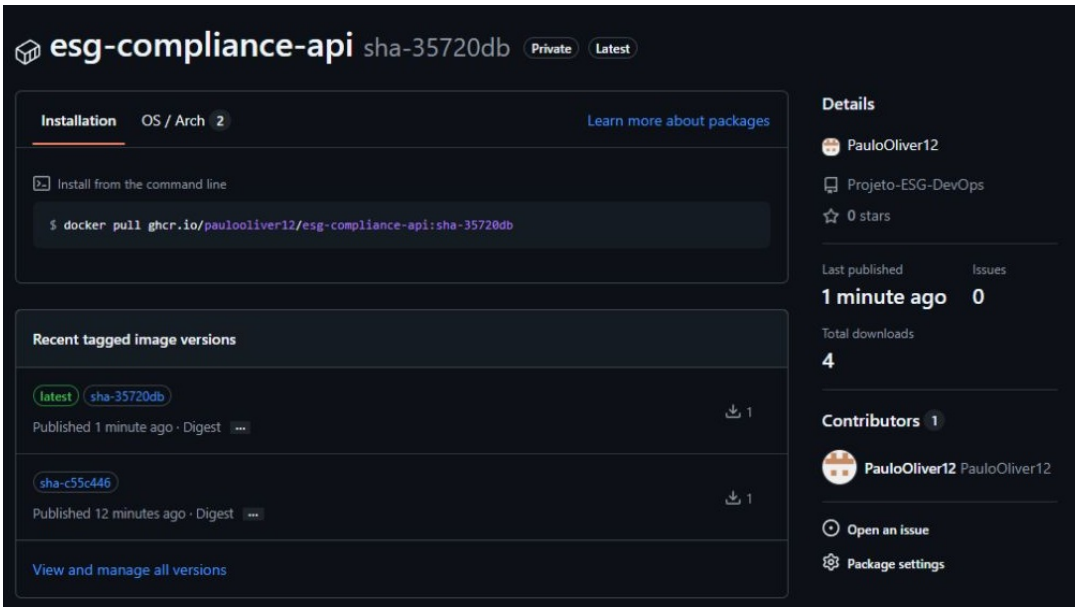



Figura 5. Pacote esg-compliance-api no GHCR, privado e com tag do commit validado. Recorte da captura; o original acompanha a entrega.

## Identidade da imagem

```
ghcr.io/paulooliver12/esg-compliance-api:sha-35720db
Digest publicado:
sha256:c4faceecbbb23245ec592a6d40dd38f8888ceec98376e8bab2619ef0103f9fdd
```

O workflow gera a imagem uma vez e passa a referência publicada aos dois deploys. Os retornos de /actuator/info em staging e production indicam a versão sha-35720db. O digest foi obtido do log de publicação e permite identificar o conteúdo registrado.

| Registro preservado | Local na entrega |
|---|---|
| Resultados Maven originais | docs/evidencias/github/surefire-reports/ |
| Health, info, containers e logs | docs/evidencias/github/staging/ e production/ |
| IDs e hashes dos artefatos | docs/evidencias/github/resultado-ci.json |
| Integridade dos arquivos do pacote | docs/manifesto-arquivos.json |

Os links do repositório e do Actions exigem acesso autorizado, pois o projeto foi mantido privado. O PDF, os prints e os registros necessários à avaliação também estão incluídos no ZIP.

08 / DESAFIOS

# Decisões e soluções

Problemas de execução, escolhas de ambiente e limites identificados.

| Situação | Tratamento adotado |
|---|---|
| Preparação do Docker | O histórico fornecido registra habilitação de virtualização e instalação do Docker Desktop. Na revisão atual, a API voltou a responder após a inicialização do ambiente. |
| Prontidão do Oracle | Healthcheck do banco e depends_on: service_healthy evitam iniciar a API antes da disponibilidade inicial do Oracle. O smoke test aguarda health UP por até 600 segundos. |
| Sem VM Azure disponível | Foi escolhido o modo runner já implementado. Os ambientes são temporários e as evidências precisam ser preservadas como artefatos do workflow. |
| Falha real no primeiro deploy | Staging retornou 401 no dashboard: Secrets vazios do job prevaleciam sobre deploy.env [R4]. O workflow passou a transmitir os valores gerados por GITHUB_ENV. A reprodução com Compose confirmou a causa; a execução #2 validou os dois deploys. |
| Rastreabilidade das evidências | Respostas HTTP locais, XML Surefire e artefatos dos deploys foram preservados. A URL do run e o commit 35720db vinculam os resultados à versão executada. |

## Limitações e melhorias futuras

O projeto não implementa rollback automático, escalonamento horizontal ou alta disponibilidade. Também não há medição de cobertura de código, teste de carga ou backup/restauração exercitado nesta entrega. Esses recursos não são inferidos da existência do pipeline.

O perfil production exige variáveis para suas senhas, mas o Compose base possui valores locais de fallback. No workflow, os valores são fornecidos por Secrets ou gerados aleatoriamente. Ao executar production fora do workflow, é necessário garantir a substituição explícita dessas credenciais.

A carga V2 é demonstrativa e contém datas fixas. O texto do alerta não é recalculado automaticamente a partir do vencimento da licença; os indicadores devem ser interpretados como dados do exemplo.

O GitHub emitiu avisos de descontinuação de versões de Actions e de futura mudança de ubuntu-latest. Eles não impediram a execução; atualizar as Actions e fixar a versão do runner são melhorias de manutenção.

09 / REPRODUÇÃO

# Como executar e comprovar

Passos para reproduzir a validação local e a execução do GitHub.

## Execução local

Com Docker Desktop ativo, abra um terminal na raiz do projeto. Copie o modelo de configuração e inicie a stack. O primeiro start do Oracle pode levar alguns minutos.

```
Copy-Item .env.example .env
docker compose up -d --build
docker compose ps
curl.exe http://localhost:8080/actuator/health
curl.exe http://localhost:8080/actuator/info
```

## Testes automatizados, sem instalar Maven no host

```
docker run --rm -v "${PWD}:/app" -w /app `
  maven:3.9.9-eclipse-temurin-21 mvn -B clean verify
```

Comando em PowerShell. Preserve target/surefire-reports/ e o trecho final BUILD SUCCESS. Os testes unitários e de controller não precisam de banco em execução.

## Publicação e deploy no runner

| Passo | Verificação necessária |
|---|---|
| 1. Repositório | Publicar os arquivos na raiz do repositório, incluindo .github/workflows; manter .env e target fora do Git. |
| 2. Ambientes | Usar staging e production. Configurar Required reviewers em production se for exigida aprovação manual e o plano permitir. |
| 3. Workflow | Fazer push na main ou usar Run workflow. Conferir as cinco etapas, sem interpretar jobs ignorados como sucesso. |
| 4. Evidências | Guardar URL e commit da execução, resumo dos testes, pacote GHCR e os artefatos evidencias-staging/production. |
| 5. Documento | Atualizar as evidências a cada mudança no código. Esta versão documenta o commit 35720db e a execução #2. |

O job de publicação já declara packages: write. Se houver bloqueio por política do repositório ou da organização, é necessário revisar as permissões efetivas. A disponibilidade de proteções dos Environments depende do plano e da visibilidade [R2].

10 / RASTREABILIDADE

# Checklist e referências

Situação comprovada em 13/10/2026, com evidências preservadas na entrega.

| Item | Situação | Onde verificar |
|---|---|---|
| Código e configuração | Conferidos | src/, pom.xml, Dockerfile e arquivos Compose |
| Integrantes | Preenchidos | Capa e README.md |
| Suíte automatizada | 21 testes aprovados | docs/evidencias/github/surefire-reports/ |
| API local / Swagger | 10 verificações HTTP OK | docs/evidencias/local/ e Figura 2 |
| Pipeline CI/CD | Cinco etapas aprovadas | Execução #2; Figura 3; resultado-ci.json |
| Imagem no GHCR | Publicada; pacote privado | Tag sha-35720db e digest registrado |
| Deploy staging e produção | Concluídos no runner | Figura 4 e artefatos por ambiente |
| Enunciado oficial | Não disponível nesta revisão | Conferência final dos critérios da atividade |

**Fechamento técnico**
A entrega demonstra o ciclo completo: compilação, testes, integração com Oracle, publicação da imagem e implantação validada em dois ambientes temporários. A aderência integral à rubrica acadêmica depende da conferência com o enunciado oficial, não disponibilizado nesta revisão.

## Referências

**Fontes do projeto:** repositório privado; GitHub Actions #2; XML Surefire, artefatos dos deploys e respostas HTTP de 13/10/2026. O trabalho anterior foi utilizado somente como referência de organização.

[R1] Docker. Control startup and shutdown order in Compose. Consultar documentação. Acesso: 13 out. 2026.

[R2] GitHub Docs. Managing environments for deployment. Consultar documentação. Acesso: 13 out. 2026.

[R3] GitHub Docs. GitHub-hosted runners. Consultar documentação. Acesso: 13 out. 2026.

[R4] Docker. Set, use, and manage variables in a Compose file with interpolation. Consultar documentação. Acesso: 13 out. 2026.